

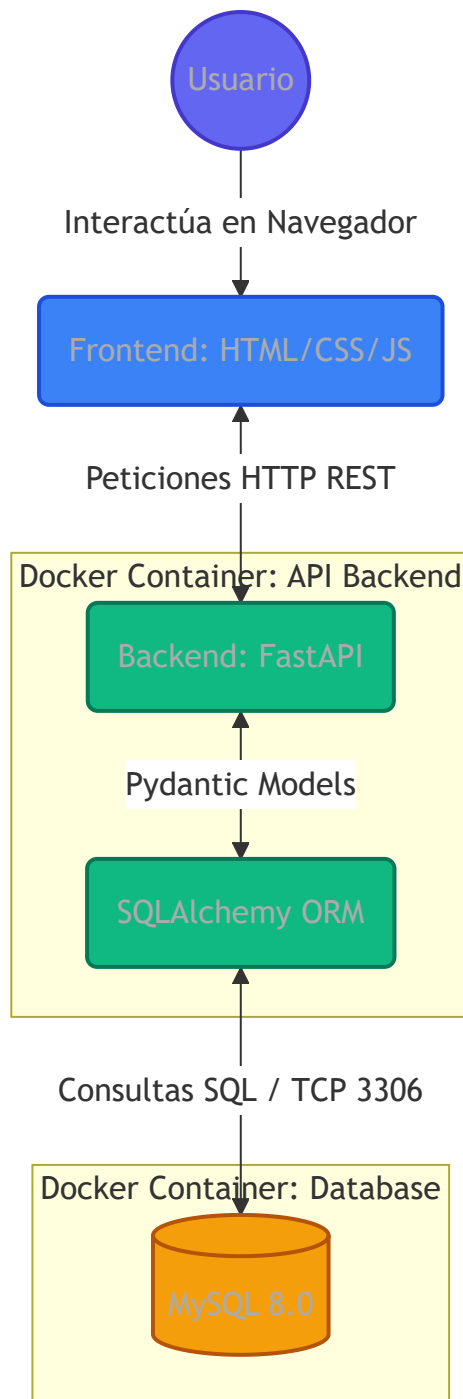
Documentación Técnica Exhaustiva - EuroCoinCollector

1. Introducción al Sistema

EuroCoinCollector no es solo un simple registro de monedas; es una plataforma integral diseñada con una arquitectura moderna de microservicios (simulada) para garantizar escalabilidad, facilidad de mantenimiento y un bajo acoplamiento entre sus componentes. Este documento detalla minuciosamente las decisiones arquitectónicas, con especial énfasis en la comunicación entre capas, la contenedorización y la implementación del paradigma REST.

2. Arquitectura General del Sistema

El proyecto ha sido diseñado siguiendo el patrón de arquitectura **Cliente-Servidor** multicapa. Hemos dividido responsabilidades de forma estricta para asegurar que cada componente se encargue únicamente de su tarea (Single Responsibility Principle).



¿Por qué lo hemos realizado de esta manera?

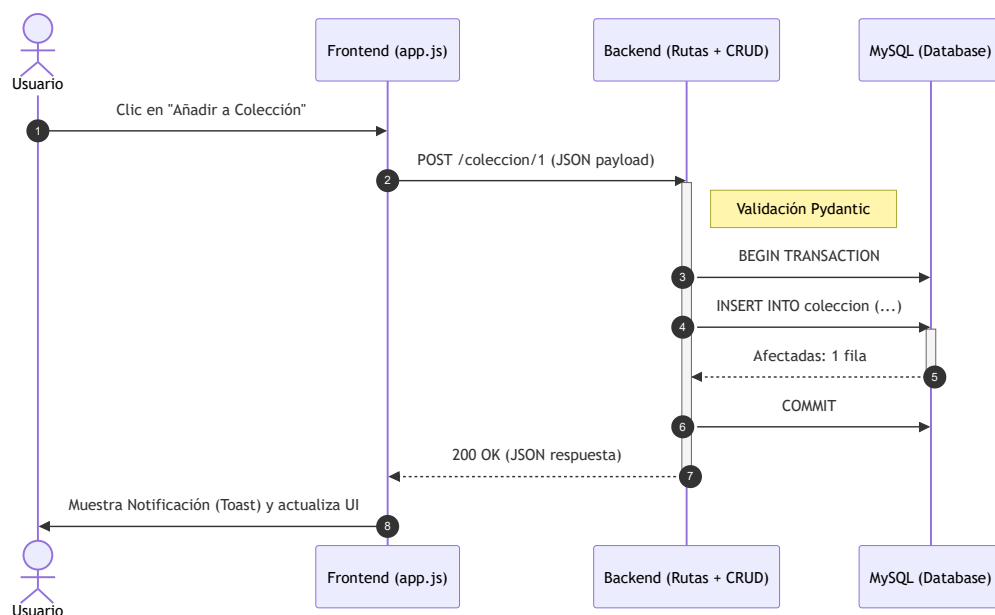
La separación estricta entre Frontend, Backend y Base de Datos permite que, en el futuro, podamos cambiar completamente la interfaz de usuario (por ejemplo, creando una app móvil en Flutter o React Native) sin tocar una sola línea de código del Backend o la Base de Datos.

3. Conexión Frontend - Backend - Base de Datos

Uno de los pilares fundamentales del proyecto es cómo fluye la información desde el clic del usuario hasta el almacenamiento persistente en el disco del servidor.

Flujo de Datos Detallado

1. **Frontend (El Cliente):** Utilizamos Vanilla JavaScript (`app.js`) empleando la API `fetch()` . Cuando el usuario realiza una acción (ej. añadir una moneda a su colección), JavaScript intercepta el evento, recolecta los datos del formulario, los convierte a un objeto JSON y realiza una petición HTTP asíncrona hacia el Backend.
2. **Backend (El Servidor):** FastAPI está escuchando en el puerto 8000. Recibe el JSON y utiliza `Pydantic` para **validar** que los tipos de datos sean correctos (ej. que la cantidad sea un número entero, que la fecha sea válida). Si es correcto, invoca una función CRUD.
3. **ORM y Base de Datos:** El ORM `SQLAlchemy` toma los datos validados y los traduce de objetos Python a una sentencia SQL (`INSERT` o `UPDATE`). Esta sentencia se envía al contenedor de MySQL a través del puerto 3306.
4. **Retorno:** MySQL confirma la inserción, el ORM actualiza el estado en Python, FastAPI serializa la respuesta a JSON y la envía de vuelta al Frontend, que actualiza el DOM dinámicamente sin recargar la página.



¿Por qué lo hemos realizado así?

- **Seguridad:** El Frontend nunca se conecta directamente a la Base de Datos. Las credenciales de MySQL solo las conoce el Backend.
 - **Validación robusta:** Pydantic evita la inyección de SQL y errores de datos sucios antes de que siquiera lleguen a la capa de base de datos.
 - **UX (Experiencia de Usuario):** Al ser asíncrono, la página nunca se "queda en blanco" ni recarga, dando la sensación de una aplicación nativa.
-

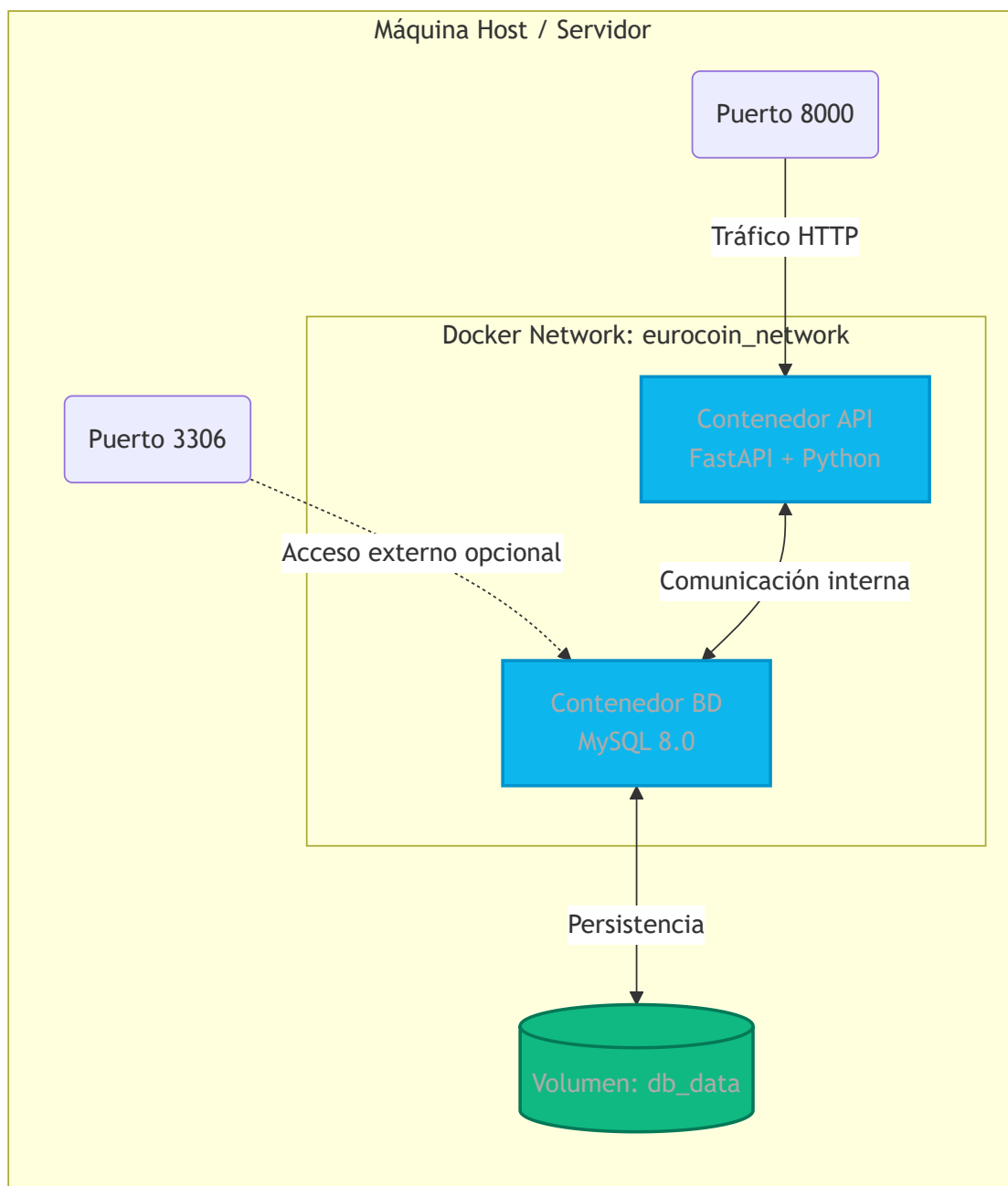
4. Encapsulamiento con Docker

Hemos utilizado Docker para "encapsular" el proyecto. Esto significa que la aplicación no depende de lo que el desarrollador tenga instalado en su ordenador (versión de Python, configuración de MySQL, etc.), sino que todo viene empaquetado en contenedores aislados.

Separación en Dos Contenedores

Hemos definido un archivo `docker-compose.yml` que orquesta dos servicios distintos:

1. **Contenedor api** : Construido a partir de nuestro propio `Dockerfile` . Contiene una imagen ligera de Python 3.11, instala nuestras dependencias (`requirements.txt`) y ejecuta el servidor Uvicorn.
2. **Contenedor db** : Utiliza la imagen oficial de MySQL 8.0.



¿Por qué lo hemos separado en dos contenedores?

- **Escalabilidad:** Si la aplicación recibe mucho tráfico, podemos clonar (replicar) el contenedor de la API detrás de un balanceador de carga, manteniendo una única base de datos. Si estuvieran en el mismo contenedor, clonar la API clonaría también la base de datos, causando conflictos masivos.
- **Persistencia Aislada:** El contenedor de MySQL tiene mapeado un *Volume* (`db_data`). Si el contenedor de la base de datos se borra o se actualiza,

los datos de los usuarios siguen a salvo en el disco duro del ordenador host.

- **Seguridad en Red Interna:** Docker crea una red virtual privada. El contenedor API puede encontrar a la base de datos simplemente llamando a la máquina `db`. Los puertos de la base de datos no necesitan estar expuestos al mundo exterior para que la API funcione.

5. La Arquitectura REST en el Proyecto

Hemos implementado una API **RESTful** (Representational State Transfer). REST no es un programa, sino un conjunto de reglas y convenciones sobre cómo debe estructurarse la comunicación web.

Principios REST Aplicados:

1. **Uso de Verbos HTTP semánticos:**
2. `GET /monedas` : Obtiene (lee) el listado de monedas.
3. `POST /lista_deseos/5` : Crea un nuevo registro para la moneda 5.
4. `PUT /coleccion/2` : Actualiza (o crea si no existe) el registro de la moneda 2.
5. `DELETE /coleccion/2` : Borra el registro.
6. **Stateless (Sin Estado):** Cada petición del frontend al backend contiene toda la información necesaria para ser procesada. El servidor no "recuerda" la petición anterior. Esto hace que el servidor sea extremadamente rápido y fácil de escalar.
7. **Identificadores de Recursos Uniformes (URI):** Nuestras rutas se centran en *sustantivos* (recursos) en plural, no en acciones. En vez de `/obtenerMonedas` o `/borrarIntercambio`, usamos `/monedas` o `/intercambios/{id}`.
8. **Respuestas Estructuradas (JSON):** Toda la comunicación de salida está codificada en JSON estándar, fácilmente interpretable por cualquier lenguaje de programación en el cliente.
9. **Códigos de Estado HTTP:**
10. `200 OK` : Cuando todo va bien.
11. `201 Created` : Al crear un intercambio.
12. `404 Not Found` : Si intentas buscar una moneda que no existe.

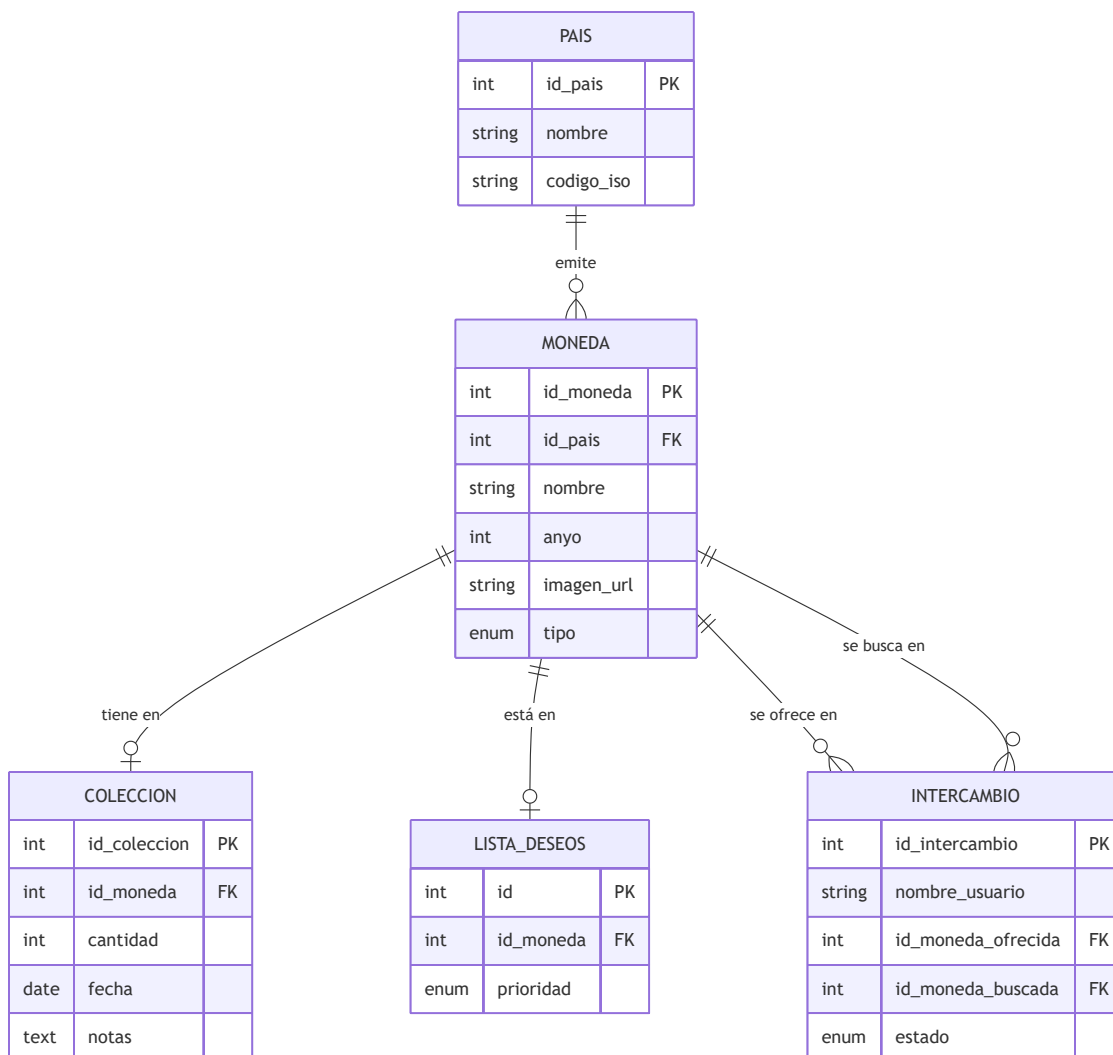
13. 422 Unprocessable Entity : Si Pydantic detecta que envías un texto donde debe ir un número.

¿Por qué REST?

Al adherirnos estrictamente a REST, nuestra API es intuitiva. Cualquier desarrollador externo que vea la ruta `DELETE /lista_deseos/10` sabrá inmediatamente qué hace, sin necesidad de leer manuales extensos. Además, es el estándar dorado en el desarrollo web moderno, asegurando compatibilidad con casi cualquier herramienta o framework.

6. Modelo Entidad-Relación y Estructura de Datos

El núcleo del sistema es su base de datos relacional. Hemos normalizado los datos para evitar redundancias.



Explicación del Modelo

- Separar PAIS de MONEDA nos permite cambiar el nombre de un país una sola vez, afectando a todas sus monedas instantáneamente (normalización).
- COLECCION y LISTA_DESEOS actúan como tablas de extensión de MONEDA . No modificamos la tabla central del catálogo para saber si un usuario tiene la moneda, sino que creamos un registro relacionado. Esto es vital para el día de mañana añadir un sistema de múltiples usuarios, donde cada usuario tendría su propio registro apuntando a la misma moneda del catálogo general.

7. Conclusión

El diseño de **EuroCoinCollector** demuestra la aplicación práctica de arquitecturas empresariales modernas. Al usar FastAPI, MySQL, VanillaJS moderno y Docker, hemos construido un sistema que es veloz, predecible, altamente testeable e inherentemente escalable.